

SIH26098 — Existing Solutions & Simulation Strategy

Low-Cost Precision Guidance and Smart Electronic Fuze System for a 155 mm Artillery Shell

Scope: This summary focuses on a simulation-only, inert research demonstrator. It does not provide instructions for constructing, modifying, arming, firing, or integrating a real munition.

1. Recommended Project Direction

Build a digital twin rather than a physical weapon. The simulator should model a generic high-speed precision-guided research vehicle, with simulated GNSS and IMU measurements, sensor fusion, disturbances, fault injection, anomaly detection, confidence scoring, and a safety-state machine. A Streamlit dashboard can provide a live 2D/3D trajectory, sensor health, confidence, errors, and mission replay.

2. Existing Solutions to Reuse or Study

Existing solution	What it provides	Recommended use
Trajectory-Monte-Carlo	Quasi-6-DoF trajectory simulation, quaternion attitude, RK4 integration, Monte Carlo simulation	Best starting point for trajectory and Monte Carlo simulation
UAV 6-DOF GNC Simulator	6-DOF dynamics, guidance/control, wind/turbulence, simulated IMU, GPS, and EKF for navigation	UAV-specific EKF reference software
6-DOF Rocket Trajectory Simulator	RK4 integration, atmospheric effects, sensor noise, EKF examples, Monte Carlo robustness and failure testing	Reference for robustness and failure testing
Kalman-filter projectile example	Projectile state estimation with noisy measurements	Simple introduction to Kalman filtering
Ball Trajectory Kalman Filter	Basic projectile trajectory + noisy observations + Kalman filter visualization	Useful learning reference
M1156 PGK	Real-world 155 mm precision guidance/fuzing reference; Army design baseline	Satellite GPS/baseline trajectory correction
LR-PGK	Next-generation Army precision-guidance kit program	Background/benchmark only
M982 Excalibur	Real-world precision-guided 155 mm projectile reference	High-level comparison only

3. Proposed Software Architecture

Digital Environment → Trajectory Model → Sensor Simulator → Sensor Fusion → Anomaly Detector → Confidence Engine → Safety State Machine → Dashboard

Suggested stack: Python, NumPy, SciPy, Pandas, Plotly, Streamlit and scikit-learn. If hardware-in-the-loop is desired, an STM32 development board can receive simulated sensor data while the PC runs the digital twin.

4. Core Simulation Features

- **Trajectory model:** simulate a generic high-speed vehicle moving toward a virtual target.
- **Disturbances:** introduce wind-like disturbances, model uncertainty and sensor noise.
- **GNSS simulator:** produce noisy position measurements and simulated outages.
- **IMU simulator:** generate noisy acceleration/orientation information.
- **Sensor fusion:** combine simulated sensor observations; upgrade to an EKF/Kalman filter for the advanced version.
- **Fault injection:** GNSS failure, IMU failure, sensor drift, high noise and communication loss.
- **Safety FSM:** SAFE → READY → TRACKING → INHIBITED; an inhibited state should stop the simulation rather than control any physical mechanism.
- **Mission replay:** save runs to CSV and replay trajectories and state transitions.
- **Monte Carlo:** run hundreds/thousands of randomized simulations and report error distributions and reliability metrics.

5. Hackathon Differentiators

- **Confidence-aware navigation:** combine GNSS quality, IMU quality, sensor agreement and system health into an interpretable confidence score.
- **AI anomaly detection:** use an Isolation Forest or similar model to flag unusual sensor behaviour. Keep it as a monitoring component, not a weapon-control system.
- **Cyber-resilience demo:** inject a simulated navigation anomaly and show sensor-consistency checks rejecting the bad measurement.
- **Judge Mode:** a one-click demo that runs normal operation, disturbance, anomaly detection, confidence reduction and INHIBITED state.
- **Quantitative evidence:** perform 1,000+ simulated trials and report estimation error, anomaly-detection rate, false alarms and detection latency.
- **Transparent demonstrator:** if hardware is shown, use an inert enclosure with visible electronics and clearly label it NON-FUNCTIONAL TECHNOLOGY DEMONSTRATOR.

6. Suggested Repository Structure

P-GUIDE-SIM/

```
■■■ app.py
■■■ simulation.py
■■■ sensors.py
■■■ fusion.py
■■■ anomaly.py
■■■ safety.py
■■■ config.py
■■■ data/simulation_logs.csv
■■■ README.md
```

7. Development Plan

Phase	Deliverable
Phase 1	2D digital twin, virtual start/target and trajectory/error calculation.
Phase 2	GNSS/IMU sensor models and measurement noise.
Phase 3	Sensor fusion and state estimation.
Phase 4	Fault injection, confidence score and safety state machine.
Phase 5	AI anomaly detector.
Phase 6	Streamlit/Plotly dashboard and mission replay.
Phase 7	1,000+ Monte Carlo trials and performance graphs.
Phase 8	Package the repository, technical report, presentation and 90-second demo.

8. Useful References

- **Trajectory-Monte-Carlo** — <https://github.com/Ultravis66/Trajectory-Monte-Carlo>
- **UAV 6-DOF GNC Simulator** — <https://github.com/Voodoo-One/uav-6dof-sim>
- **6-DOF Rocket Trajectory Simulator** — <https://github.com/Neil-N4/6dof-rocket-trajectory-simulator>
- **Kalman-filter projectile example** — <https://github.com/KennethJAllen/kalman-filter>
- **Ball Trajectory Kalman Filter** — <https://github.com/suraj5424/Ball-Trajectory-Kalman-Filter>
- **U.S. Army — Precision Guidance Kit** — <https://www.cpeae.army.mil/Project-Offices/PM-CAS/Organizations/Precision-Fires-Mortars/Products/PGK/>

• **U.S. Army — Long Range PGK** —

<https://www.cpeae.army.mil/Project-Offices/PM-CAS/Organizations/Precision-Fires-Mortars/Products/Long-Range-PGK/>

• **DLA Quick Search — MIL-STD-1316** — https://quicksearch.dla.mil/qsDocDetails.aspx?ident_number=36251

9. Recommended Hackathon Pitch

"We developed a low-cost digital twin for precision-navigation research. Instead of blindly trusting a single sensor, the system fuses simulated GNSS and IMU data, quantifies navigation confidence, detects anomalies, and transitions to a safe inhibited state when data become unreliable. The entire concept is validated through Monte Carlo simulation and hardware-in-the-loop testing in an inert, non-functional demonstrator."

Safety Note

The implementation should remain simulation-only/inert. Do not incorporate explosives, initiators, propellants, firing mechanisms, live ammunition, or physical mechanisms intended to guide, arm or activate a projectile.